

# NiceGUI 中 ui.add\_css 全面详解（基于官方文档）

`ui.add_css` 是 NiceGUI 中**核心的样式注入工具**，用于向应用中添加自定义 CSS 样式（支持原生 CSS、Tailwind CSS 语法），适配全局样式、组件专属样式、第三方样式等多种场景，是替代 `ui.add_head_html('<style>...</style>')` 的更简洁、更具语义化的 API。以下结合官方文档，从**基础用法**、**核心特性**、**场景化实战**、**参数详解**、**注意事项**五个维度，全面拆解其使用逻辑（覆盖文档所有内容，补充实操细节与避坑点）。

## 一、核心定位（官方核心说明）

`ui.add_css` 的核心作用是「便捷注入 CSS 样式」，本质是简化了手动编写 `<style>` 标签并通过 `ui.add_head_html` 注入的流程，同时兼容 NiceGUI 的组件模型、Tailwind CSS 集成，以及样式优先级控制，支持：

- 注入原生 CSS 样式；
- 注入 Tailwind CSS 样式（含 `@layer`、`@apply` 等语法）；
- 控制样式作用域（全局 / 组件局部）；
- 加载外部 CSS 文件；
- 与 NiceGUI 组件样式（`.classes()`、`.style()`）协同工作。

官方强调：`ui.add_css` 是 NiceGUI 推荐的样式注入方式，比直接注入 `<style>` 标签更规范、更易维护，且能避免样式加载顺序导致的失效问题。

## 二、基础用法（官方示例 + 补充扩展）

### （一）最简洁用法：直接传入 CSS 字符串（全局生效）

这是最常用的基础用法，无需额外参数，直接传入 CSS 代码，默认注入到页面 `<head>` 中，全局生效。

```
from nicegui import ui

# 基础示例：注入原生 CSS，修改标签、按钮样式（全局）
ui.add_css('''
    label {
        color: #2c3e50;
        font-size: 16px;
        margin-bottom: 8px;
    }
    .nicegui-button {
        background-color: #3498db;
        border-radius: 6px;
        padding: 8px 16px;
    }
''')

# 应用样式（组件会自动继承全局 CSS）
ui.label("全局样式：深色文本、16px 字号")
ui.button("全局样式：蓝色按钮", on_click=lambda: print("点击"))
```

```
ui.run()
```

核心说明（对应官方文档）：

- 传入的 CSS 字符串会被自动包裹在 `<style>` 标签中，注入到页面 `<head>`；
- 样式默认「全局生效」，即所有匹配的 HTML 元素、NiceGUI 组件都会被作用到；
- NiceGUI 组件的默认类名可用于样式匹配（如按钮默认类名 `nicegui-button`、输入框 `nicegui-input`），可通过浏览器开发者工具查看组件真实类名。

(二) 关键参数详解（官方核心参数 + 实操说明）

`ui.add_css` 支持多个参数，用于控制样式的加载方式、作用域、优先级，所有参数均来自官方文档，补充参数用法、默认值、适用场景：

| 参数名                 | 类型                | 默认值                     | 官方作用                                                                                       | 实操说明（避坑重点）                                                                                                                                                                                                                                                |
|---------------------|-------------------|-------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>css</code>    | <code>str</code>  | 必传                      | 要注入的 CSS 代码（原生 CSS 或 Tailwind CSS）                                                         | 1. 支持多行字符串（推荐），无需手动加 <code>&lt;style&gt;</code> 标签；2. 可混合原生 CSS 和 Tailwind 语法（如同时写 <code>background: red;</code> 和 <code>@apply bg-blue-500;</code> ）；3. 空字符串会被忽略，不会注入无效样式                                                                                |
| <code>target</code> | <code>str</code>  | <code>"head"</code>     | 样式注入目标位置，可选值： <code>"head"</code> （页面头部）、 <code>"body"</code> （页面主体）                       | 1. 推荐用默认值 <code>"head"</code> ：样式加载优先级高，避免被组件内联样式覆盖；2. <code>"body"</code> 仅用于需要在页面渲染后加载的样式（如覆盖第三方组件样式），慎用（可能导致样式闪烁）；3. 官方不推荐修改此参数，除非有特殊需求                                                                                                                |
| <code>scoped</code> | <code>bool</code> | <code>False</code>      | 控制样式作用域： <code>False</code> （全局）、 <code>True</code> （局部，仅作用于当前组件树）                         | 1. 核心重点（官方高频强调）： <code>scoped=True</code> 时，样式仅作用于「调用 <code>ui.add_css</code> 时所在的组件上下文」（即当前 <code>with</code> 块内的组件）；2. 局部样式不会污染全局，适合组件封装（如自定义卡片、弹窗的专属样式）；3. 局部样式的优先级高于全局样式，但低于组件 <code>.style()</code> 行内样式                                             |
| <code>type</code>   | <code>str</code>  | <code>"text/css"</code> | 样式类型，可选值： <code>"text/css"</code> （原生 CSS）、 <code>"text/tailwindcss"</code> （Tailwind CSS） | 1. 关键避坑点：若要使用 Tailwind 语法（ <code>@layer</code> 、 <code>@apply</code> 、Tailwind 类名），必须设置 <code>type="text/tailwindcss"</code> ；2. 默认 <code>"text/css"</code> 仅支持原生 CSS，写 Tailwind 语法会失效；3. 与 <code>@layer</code> 、 <code>tailwind.config</code> 兼容（后续实战会讲） |
| <code>media</code>  | <code>str</code>  | <code>""</code>         | 媒体查询条件（用于响应式样式，如 <code>"(max-width: 768px)"</code> ）                                       | 1. 用于适配不同屏幕尺寸，仅在满足媒体条件时，样式才生效；2. 示例： <code>media="(max-width: 768px)"</code> 表示仅在小屏（手机）上生效；3. 可与 Tailwind 响应式前缀（ <code>sm:</code> 、 <code>md:</code> ）配合使用，增强响应式能力                                                                                        |
| <code>id</code>     | <code>str</code>  | <code>None</code>       | 给注入的 <code>&lt;style&gt;</code> 标签添加唯一 ID，用于后续删除 / 替换样式                                    | 1. 用于动态管理样式（如切换主题时，删除旧样式、添加新样式）；2. 示例： <code>ui.add_css(css=..., id="custom-theme")</code> ，后续可通过 <code>ui.remove_css("custom-theme")</code> 删除；3. 若不指定，NiceGUI 会自动生成唯一 ID，无需手动设置                                                                         |

(三) 加载外部 CSS 文件（官方进阶用法）

`ui.add_css` 支持通过 `css` 参数传入外部 CSS 文件的 URL，实现外部样式文件的加载（无需手动写 `<link>` 标签），适用于加载第三方 CSS 库、自定义全局样式文件。

```
from nicegui import ui
```

```
# 示例1: 加载 CDN 上的外部 CSS (如 Font Awesome 图标样式)
ui.add_css("https://cdn.bootcdn.net/ajax/libs/font-awesome/6.4.0/css/all.min.css")

# 示例2: 加载本地外部 CSS 文件 (项目根目录下的 static/custom.css)
# 注意: 本地文件需放在 NiceGUI 可访问的静态目录 (默认 static/), 路径用相对路径或绝对路径
ui.add_css("/static/custom.css") # 推荐: 通过静态目录访问

# 应用外部样式 (如 Font Awesome 图标)
ui.label('<i class="fas fa-user"></i> 加载外部图标样式').classes("text-xl")
ui.label("加载本地 custom.css 中的样式").classes("custom-local-class") # 来自本地 CSS 文件

ui.run()
```

#### 官方注意事项补充:

1. 加载本地 CSS 文件时, 文件必须放在 NiceGUI 的「静态文件目录」(默认是项目根目录下的 `static/` 文件夹), 否则会报 404 错误;
2. 外部 CSS 文件的加载顺序: 先加载 `ui.add_css` 注入的外部文件, 再加载内部 CSS 代码, 优先级: 内部 CSS > 外部 CSS;
3. 若外部文件加载失败 (如 CDN 失效), 样式会不生效, 可添加 `on_load` 回调监听加载状态 (进阶用法)。

## 三、场景化实战 (官方示例 + 项目实操, 覆盖高频场景)

结合官方文档示例, 补充 NiceGUI 项目中最常用的 4 个场景, 每个场景均对应真实开发需求, 兼容 Tailwind CSS 集成 (衔接上一轮 Tailwind 用法)。

### 场景 1: 注入 Tailwind CSS 样式 (结合 @layer、@apply)

官方明确: `ui.add_css` 完全兼容 Tailwind CSS, 只需设置 `type="text/tailwindcss"`, 即可使用 `@layer`、`@apply` 等 Tailwind 语法, 替代手动注入 `<style type="text/tailwindcss">`。

```
from nicegui import ui

# 示例: 用 ui.add_css 注入 Tailwind 自定义类 (替代 add_head_html)
ui.add_css('''
    @layer components {
        .custom-blue-card {
            @apply bg-blue-500 text-white p-6 rounded-xl shadow-lg hover:bg-blue-600
            transition-colors;
        }
        .custom-gray-btn {
            @apply bg-gray-200 text-gray-800 px-4 py-2 rounded-md hover:bg-gray-300;
        }
    }
''', type="text/tailwindcss") # 必须设置 type, 否则 Tailwind 语法失效

# 应用自定义 Tailwind 类
with ui.column():
    ui.label("Tailwind 自定义卡片").classes("custom-blue-card")
    ui.button("Tailwind 自定义按钮").classes("custom-gray-btn mt-4")
```

```
ui.run()
```

### 官方重点强调：

- 必须设置 `type="text/tailwindcss"`，否则 `@layer`、`@apply` 会被当作原生 CSS 解析，样式完全失效；
- 此用法与 `ui.add_head_html('<style type="text/tailwindcss">...</style>')` 效果一致，但更简洁、易维护；
- 支持扩展 Tailwind 主题（如自定义颜色、间距），只需在 CSS 前注入 `tailwind.config`（后续补充）。

## 场景 2：局部样式（scoped=True，组件封装必备）

官方推荐：封装自定义组件（如卡片、弹窗）时，用 `scoped=True` 实现局部样式，避免污染全局样式，这是 NiceGUI 组件封装的核心技巧。

```
from nicegui import ui

# 示例：封装一个自定义卡片组件，样式仅作用于当前卡片（局部生效）
with ui.card() as custom_card:
    # 局部 CSS：仅作用于这个 card 内的所有组件
    ui.add_css('''
        label {
            color: #27ae60;
            font-weight: bold;
        }
        .nicegui-button {
            background-color: #f39c12;
            color: white;
        }
    ''', scoped=True) # 关键：scoped=True 局部生效

    ui.label("局部样式：绿色文本、加粗")
    ui.button("局部样式：橙色按钮", on_click=lambda: print("点击")).classes("mt-2")

# 全局组件：不受上面局部样式影响（验证局部作用域）
ui.label("全局样式：默认文本颜色（不受局部样式影响）").classes("mt-4")
ui.button("全局样式：默认按钮样式").classes("mt-2")

ui.run()
```

### 实操细节（官方补充）：

1. `scoped=True` 时，样式的作用域是「当前组件树」，即 `with ui.xxx()` 块内的所有子组件，父组件、其他组件不受影响；
2. 局部样式中，若要匹配 NiceGUI 组件，仍可使用组件默认类名（如 `nicegui-button`），或自定义类名；
3. 局部样式优先级 > 全局样式，若局部样式与全局样式冲突，以局部样式为准。

## 场景 3：动态管理样式（添加 / 删除，适配主题切换）

结合 `id` 参数，实现样式的动态添加、删除，适用于主题切换、夜间模式等场景（官方进阶示例）。

```
from nicegui import ui

# 1. 注入默认主题（全局，带唯一 ID）
ui.add_css('''
    body {
        @apply bg-white text-gray-800;
    }
    .custom-btn {
        @apply bg-blue-500 text-white;
    }
''', type="text/tailwindcss", id="light-theme") # 唯一 ID：用于后续删除

# 2. 夜间主题 CSS（未初始注入，点击按钮后添加）
dark_css = '''
    body {
        @apply bg-gray-900 text-white;
    }
    .custom-btn {
        @apply bg-purple-500 text-white;
    }
'''

# 3. 主题切换逻辑
def toggle_theme():
    if ui.get_css("light-theme"): # 判断默认主题是否存在
        ui.remove_css("light-theme") # 删除默认主题
        ui.add_css(dark_css, type="text/tailwindcss", id="dark-theme") # 添加夜间主题
        theme_label.text = "当前主题：夜间模式"
    else:
        ui.remove_css("dark-theme") # 删除夜间主题
        ui.add_css(..., type="text/tailwindcss", id="light-theme") # 恢复默认主题（省略重复 CSS）
        theme_label.text = "当前主题：浅色模式"

# 应用组件
theme_label = ui.label("当前主题：浅色模式").classes("text-xl")
ui.button("切换主题", on_click=toggle_theme).classes("custom-btn mt-2")

ui.run()
```

### 官方 API 补充：

- `ui.get_css(id: str)`：根据 ID 获取已注入的 CSS 样式，返回样式对象（无此 ID 则返回 None）；
- `ui.remove_css(id: str)`：根据 ID 删除已注入的 CSS 样式（必须传入 `id`，无法删除无 ID 的样式）；
- 动态切换样式时，建议给每个主题设置唯一 `id`，避免样式冲突。

## 场景 4：覆盖 Tailwind 默认样式（衔接上一轮需求）

结合 `type="text/tailwindcss"`，用 `ui.add_css` 覆盖 Tailwind 的默认重置样式（如 `h1-h6` 字体、`body` 背景），比 `ui.add_head_html` 更简洁。

```
from nicegui import ui

# 用 ui.add_css 覆盖 Tailwind 默认样式（无需手动写 <style> 标签）
ui.add_css('''
    /* 覆盖 h2 默认样式（Tailwind 默认重置了 h2 字体大小） */
    h2 {
        @apply text-3xl font-bold text-green-700 mb-4;
    }
    /* 覆盖 body 全局样式（Tailwind 默认 body 无背景色） */
    body {
        @apply bg-gray-50;
    }
    /* 覆盖 Tailwind 内置类（如按钮默认样式） */
    .bg-blue-500 {
        background-color: #0071e3; /* 自定义蓝色，覆盖 Tailwind 默认 bg-blue-500 */
    }
''', type="text/tailwindcss") # 必须设置 type，否则被 Tailwind 覆盖

# 渲染原生 h2 标签（sanitize=False 允许原生 HTML）
ui.html('<h2>自定义 H2 标题（覆盖 Tailwind 默认）</h2>', sanitize=False)
# 应用覆盖后的 Tailwind 类
ui.button("自定义蓝色按钮", on_click=lambda: print("点击")).classes("bg-blue-500 text-white px-4 py-2")

ui.run()
```

### 官方核心提醒：

- 覆盖 Tailwind 默认样式时，必须设置 `type="text/tailwindcss"`，否则自定义样式会被 Tailwind 的默认重置样式覆盖；
- 可直接覆盖 Tailwind 内置类（如 `bg-blue-500`），也可覆盖 HTML 原生元素样式（如 `h2`、`body`）；
- 优先级：`ui.add_css(type="text/tailwindcss")` > Tailwind 默认样式 > 原生 CSS 全局样式。

## 四、与其他样式 API 的对比（官方说明 + 实操选型）

NiceGUI 中还有 `.classes()`、`.style()`、`ui.add_head_html` 等样式相关 API，结合官方文档，明确 `ui.add_css` 的适用场景，避免混淆：

| API                     | 核心作用                         | 与 ui.add_css 的区别                                       | 适用场景                                                   |
|-------------------------|------------------------------|--------------------------------------------------------|--------------------------------------------------------|
| <code>ui.add_css</code> | 注入全局 / 局部 CSS（原生 / Tailwind） | 1. 批量注入样式，支持全局 / 局部控制；2. 兼容 Tailwind；3. 可动态管理（添加 / 删除） | 1. 全局样式配置；2. 组件封装的局部样式；3. 加载外部 CSS；4. 覆盖 Tailwind 默认样式 |

| API                                                             | 核心作用                      | 与 ui.add_css 的区别                                                                                                                | 适用场景                                            |
|-----------------------------------------------------------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <code>.classes()</code>                                         | 给单个组件添加 Tailwind 类 / 自定义类 | 1. 仅作用于单个组件；2. 简洁易用，适合快速设置组件样式；3. 无法写原生 CSS                                                                                     | 单个组件的样式快速设置（如按钮、标签的颜色、间距）                       |
| <code>.style()</code>                                           | 给单个组件添加行内样式               | 1. 行内样式，优先级最高（覆盖所有 CSS）；2. 仅作用于单个组件；3. 写原生 CSS 语法                                                                               | 单个组件的特殊样式（如自定义定位、特殊颜色，无法用 Tailwind 实现）          |
| <code>ui.add_head_html('&lt;style&gt;...&lt;/style&gt;')</code> | 手动注入样式标签                  | 1. 语法繁琐，需手动写 <code>&lt;style&gt;</code> ；2. 无 <code>scoped</code> 、 <code>id</code> 等参数，难以动态管理；3. 优先级低于 <code>ui.add_css</code> | 兼容旧版代码，或需要注入特殊样式标签（如 <code>@import</code> 外部样式） |

官方选型建议（重点）：

- 优先用 `ui.add_css` 管理全局样式、局部样式、Tailwind 自定义样式；
- 单个组件的简单样式，用 `.classes()`（结合 Tailwind）；
- 单个组件的特殊样式，用 `.style()`；
- 避免用 `ui.add_head_html` 注入样式，除非有特殊需求。

五、官方注意事项与避坑总结（必看）

1. 样式优先级（官方明确顺序，从高到低）：
  - 组件 `.style()` 行内样式 → `ui.add_css(scoped=True)` 局部样式 → `ui.add_css(scoped=False)` 全局样式 → Tailwind 默认样式 → 浏览器默认样式；
  - 可使用 `!important` 强制提升样式优先级（如 `background: red !important;`），但官方不推荐（易导致样式混乱）。
2. Tailwind 兼容问题（高频避坑）：
  - 若要使用 Tailwind 语法（`@layer`、`@apply`、Tailwind 类名），必须设置 `type="text/tailwindcss"`；
  - `scoped=True` 与 Tailwind 兼容，局部 Tailwind 样式仅作用于当前组件树；
  - 扩展 Tailwind 主题（如自定义颜色）时，可在 `ui.add_css` 前注入 `tailwind.config`（通过 `ui.add_head_html` 注入 `<script>` 标签）。
3. 局部样式（`scoped=True`）注意点：
  - 局部样式仅作用于「调用 `ui.add_css` 时所在的组件上下文」，若在组件外调用，`scoped=True` 无效（仍全局生效）；
  - 局部样式中，无法通过 `body`、`html` 选择器修改全局样式（仅能修改当前组件树内的元素）。
4. 外部 CSS 加载注意点：

- 本地外部 CSS 文件必须放在 `static/` 目录下，访问路径用 `/static/xxx.css`；
- 加载 CDN 外部 CSS 时，需确保 CDN 链接有效，否则样式失效；
- 外部 CSS 加载顺序晚于内部 CSS，若冲突，以内部 CSS 为准。

#### 5. 动态样式管理注意点：

- 只有设置了 `id` 的样式，才能通过 `ui.remove_css`、`ui.get_css` 管理；
- 多次注入相同 `id` 的样式，会覆盖之前的样式（无需手动删除）。

## 六、进阶扩展（官方文档延伸，适配项目开发）

### （一）扩展 Tailwind 主题（结合 `ui.add_css`）

若需自定义 Tailwind 主题（如自定义颜色、字体、间距），可先注入 `tailwind.config`，再用 `ui.add_css` 使用自定义主题样式：

```
from nicegui import ui

# 1. 注入自定义 Tailwind 配置（扩展主题）
ui.add_head_html('''
<script>
    tailwind.config = {
        theme: {
            extend: {
                colors: {
                    primary: '#165DFF', // 自定义主色调
                },
                spacing: {
                    '18': '4.5rem', // 自定义间距
                }
            }
        }
    }
</script>
''')

# 2. 用 ui.add_css 注入自定义 Tailwind 类（使用扩展后的主题）
ui.add_css('''
@layer components {
    .custom-primary-btn {
        @apply bg-primary text-white px-6 py-2 rounded-lg; // 使用自定义颜色 primary
    }
}
''', type="text/tailwindcss")

# 3. 应用样式
ui.button("自定义主色调按钮").classes("custom-primary-btn mt-4")
ui.label("自定义间距").classes("mt-18") # 使用自定义间距 18

ui.run()
```

## （二）结合组件生命周期使用

在组件挂载、卸载时，动态添加 / 删除样式，适配弹窗、抽屉等组件：

```
from nicegui import ui

def open_drawer():
    with ui.drawer() as drawer:
        # 抽屉挂载时，注入局部样式
        ui.add_css('''
            .nicegui-label {
                color: #e74c3c;
                font-size: 18px;
            }
        ''', scoped=True)
        ui.label("抽屉内局部样式：红色文本、18px 字号")
        # 抽屉卸载时，删除样式（可选，scoped 样式会随组件卸载自动删除）
        drawer.on_unmount(lambda: ui.remove_css(drawer.style_id)) # 假设给样式设置了 id

ui.button("打开抽屉", on_click=open_drawer)
ui.run()
```

## 七、总结（官方核心要点提炼）

`ui.add_css` 是 NiceGUI 样式管理的「核心 API」，核心优势的是「简洁、灵活、兼容 Tailwind」，总结 5 个核心要点（适配项目开发）：

1. 基础用法：直接传入 CSS 字符串，默认全局生效，简化样式注入流程；
2. 核心参数：`scoped`（控制作用域）、`type`（适配 Tailwind）、`id`（动态管理）是必掌握的参数；
3. 核心兼容：完美支持 Tailwind CSS，设置 `type="text/tailwindcss"` 即可使用所有 Tailwind 语法；
4. 场景选型：全局样式、组件局部样式、外部 CSS 加载、主题切换，优先用 `ui.add_css`；
5. 避坑关键：Tailwind 样式必须设 `type="text/tailwindcss"`，局部样式需在组件上下文内调用 `scoped=True`。

## NiceGUI 中 `ui.add_css` 完全指南（基于官方文档）

`ui.add_css()` 是 NiceGUI 提供的**原生 CSS 注入工具**，用于向应用中添加自定义 CSS 样式，与 Tailwind CSS 用法（需指定 `text/tailwindcss` 类型）形成互补——前者专注原生 CSS，后者专注 Tailwind 语法。以下基于 [NiceGUI 官方文档](#)，从**核心用法**、**参数详解**、**示例场景**、**注意事项**、**与 Tailwind 对比**等维度，全面拆解 `ui.add_css()` 的使用。

# 一、核心概述

## 1. 功能定位

- 向 NiceGUI 应用的 `<head>` 标签中注入原生 CSS 样式（无需手动写 `<style>` 标签）；
- 支持内联 CSS 字符串、外部 CSS 文件、CSS 片段三种注入方式；
- 样式作用于全局，可覆盖组件默认样式、自定义组件外观，或实现 Tailwind 难以覆盖的精细样式。

## 2. 基础语法

```
from nicegui import ui

# 语法1: 注入CSS字符串
ui.add_css("body { background-color: #f0f0f0; }")

# 语法2: 注入外部CSS文件
ui.add_css(path="path/to/your/style.css")

# 语法3: 注入CSS片段（带选择器、媒体查询等完整语法）
ui.add_css("""
    .custom-btn {
        padding: 8px 16px;
        border-radius: 4px;
        border: none;
    }
    .custom-btn:hover {
        opacity: 0.9;
    }
""")

ui.run()
```

# 二、参数详解（官方文档核心参数）

`ui.add_css()` 的参数设计简洁但覆盖核心场景，所有参数均为可选（除核心的 CSS 内容 / 路径外），具体如下：

| 参数名                | 类型               | 说明                                                                                                | 官方示例                                                                |
|--------------------|------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <code>css</code>   | <code>str</code> | 要注入的 <b>原生 CSS 字符串</b> （与 <code>path</code> 二选一，优先 <code>path</code> ）                            | <code>ui.add_css("h1 { color: red; }")</code>                       |
| <code>path</code>  | <code>str</code> | 外部 CSS 文件的 <b>路径</b> （绝对路径或相对于主脚本的相对路径）                                                           | <code>ui.add_css(path="styles/main.css")</code>                     |
| <code>media</code> | <code>str</code> | 媒体查询条件（用于响应式 CSS，如 <code>print</code> 、 <code>screen</code> and <code>(max-width: 768px)</code> ） | <code>ui.add_css("body { font-size: 12pt; }", media="print")</code> |

| 参数名                 | 类型                | 说明                                                                                                     | 官方示例                                                                        |
|---------------------|-------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>scoped</code> | <code>bool</code> | 是否作用域隔离（默认 <code>False</code> ）：- <code>False</code> ：样式全局生效；- <code>True</code> ：样式仅作用于当前页面（多页面应用中有用） | <code>ui.add_css(".box { color: blue; }", scoped=True)</code>               |
| <code>id</code>     | <code>str</code>  | 为注入的 <code>&lt;style&gt;</code> 标签添加 <code>id</code> 属性（用于后续通过 JS 操作样式）                                | <code>ui.add_css("p { margin: 10px; }", id="custom-paragraph-style")</code> |

## 关键参数补充（官方文档隐含细节）

- 1. `css` 与 `path` 优先级：  
若同时指定 `css` 和 `path`，`path` 会覆盖 `css`（即优先加载外部文件，忽略内联 CSS 字符串）。
- 2. `scoped` 的生效范围：  
仅在多页面应用（如使用 `ui.page` 定义多个页面）中有效，单页面应用中 `scoped=True` 无意义（样式仍全局生效）。
- 3. `media` 的用法：  
支持所有 CSS 原生媒体查询语法，常用于「打印样式」「移动端适配」等场景，与原生 CSS 的 `@media` 等价。

## 三、典型使用场景（结合官方示例 + 实战扩展）

### 场景 1：注入简单内联 CSS（快速自定义全局样式）

适用于少量、临时的样式调整，无需创建外部文件。

```
from nicegui import ui

# 自定义全局文本样式和按钮样式
ui.add_css("""
    /* 全局文本样式 */
    body {
        font-family: 'Arial', sans-serif;
        line-height: 1.6;
        color: #333;
    }
    /* 自定义按钮样式 */
    .q-btn { /* NiceGUI 按钮的底层类是 .q-btn */
        background-color: #4CAF50 !important; /* !important 覆盖默认样式 */
        color: white !important;
        padding: 8px 16px !important;
        border-radius: 4px !important;
    }
    .q-btn:hover {
        background-color: #45a049 !important;
    }
""")
```

```
ui.label("全局字体已修改为 Arial")
ui.button("自定义绿色按钮").classes("mt-4") # 按钮会应用上面的 .q-btn 样式

ui.run()
```

## 场景 2：加载外部 CSS 文件（规范管理样式）

适用于样式较多的项目，将 CSS 单独放在文件中，便于维护。

### 步骤 1：创建外部 CSS 文件（如 `styles/book_system.css`）

```
/* 图书管理系统专用样式 */
/* 卡片样式 */
.book-card {
    border: 1px solid #e0e0e0;
    border-radius: 8px;
    padding: 16px;
    margin: 8px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
/* 标题样式 */
.book-title {
    font-size: 18px;
    font-weight: bold;
    color: #2c3e50;
    margin-bottom: 8px;
}
/* 响应式调整（小屏隐藏） */
@media (max-width: 600px) {
    .book-card {
        margin: 4px;
        padding: 12px;
    }
}
```

### 步骤 2：在 NiceGUI 中加载外部 CSS

```
from nicegui import ui

# 加载外部CSS文件（相对路径：主脚本所在目录下的 styles 文件夹）
ui.add_css(path="styles/book_system.css")

# 应用外部CSS中的类
with ui.row():
    with ui.column().classes("book-card"): # 应用 .book-card 样式
        ui.label("Python编程：从入门到实践").classes("book-title") # 应用 .book-title 样式
        ui.label("作者：埃里克·马瑟斯")
    with ui.column().classes("book-card"):
        ui.label("SQLAlchemy实战").classes("book-title")
        ui.label("作者：杰森·迈尔斯")
```

```
ui.run()
```

## 场景 3：媒体查询（适配打印 / 移动端）

使用 `media` 参数指定样式的适用场景，例如「打印时隐藏按钮，调整字体」。

```
from nicegui import ui

# 注入打印专用样式（仅打印时生效）
ui.add_css("""
    /* 打印时隐藏按钮 */
    .q-btn {
        display: none !important;
    }
    /* 打印时调整标题样式 */
    h1 {
        font-size: 24pt;
        color: #000;
    }
    """, media="print")

# 注入移动端专用样式（屏幕宽度≤768px时生效）
ui.add_css("""
    h1 {
        font-size: 18px;
    }
    .container {
        padding: 8px;
    }
    """, media="screen and (max-width: 768px)")

ui.html("<h1>图书管理系统报表</h1>")
ui.button("打印报表", on_click=lambda: ui.run_javascript("window.print()")).classes("mt-4")
ui.divider()
ui.label("报表内容: ...").classes("container")

ui.run()
```

## 场景 4：作用域隔离（多页面应用）

在多页面应用中，使用 `scoped=True` 确保样式仅作用于当前页面，避免页面间样式冲突。

```
from nicegui import ui

# 页面1：首页（红色标题样式，仅作用于当前页面）
@ui.page("/")
def index_page():
    ui.add_css("h1 { color: red; }", scoped=True)
    ui.html("<h1>图书管理系统首页</h1>")
    ui.link("进入图书列表", "/books")

# 页面2：图书列表（蓝色标题样式，仅作用于当前页面）
```

```
@ui.page("/books")
def books_page():
    ui.add_css("h1 { color: blue; }", scoped=True)
    ui.html("<h1>图书列表</h1>")
    ui.link("返回首页", "/")

ui.run()
```

- 访问 `/` 时，标题为红色；访问 `/books` 时，标题为蓝色；
- 若不设置 `scoped=True`，两个页面的 `h1` 样式会冲突（后加载的页面样式覆盖前一个）。

## 场景 5：结合组件 ID 精准定位样式

使用 `id` 参数为 `<style>` 标签添加标识，可后续通过 JavaScript 动态修改样式（官方文档隐含用法）。

```
from nicegui import ui

# 注入带ID的CSS样式
ui.add_css("""
    .dynamic-box {
        width: 200px;
        height: 200px;
        background-color: yellow;
    }
""", id="box-style")

ui.div().classes("dynamic-box").props("id='my-box'")
ui.button("修改背景色为红色", on_click=lambda: ui.run_javascript("""
    // 通过ID获取样式标签，修改内容
    document.getElementById('box-style').textContent = `
        .dynamic-box {
            width: 200px;
            height: 200px;
            background-color: red;
        }
    `;
"""))

ui.run()
```

## 四、与 Tailwind CSS 的区别与配合（关键！）

在 NiceGUI 中，`ui.add_css()`（原生 CSS）与 Tailwind CSS（通过 `ui.add_head_html` 注入）是**互补关系**，需明确二者的区别，避免混淆：

| 特性 | <code>ui.add_css()</code> | Tailwind CSS (@layer 等)       |
|----|---------------------------|-------------------------------|
| 语法 | 原生 CSS 语法                 | Tailwind 专用语法 (@apply、响应式前缀等) |

| 特性   | ui.add_css()                                                               | Tailwind CSS (@layer 等)                                                                      |
|------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| 注入方式 | 直接调用 <code>ui.add_css()</code> ，无需手动写 <code>&lt;style&gt;</code> 标签        | 需通过 <code>ui.add_head_html()</code> 注入 <code>&lt;style type="text/tailwindcss"&gt;</code> 标签 |
| 核心用途 | 1. 实现 Tailwind 难以覆盖的精细样式（如复杂动画、自定义伪元素）；2. 加载外部 CSS 文件；3. 多页面样式隔离；4. 媒体查询适配 | 1. 快速应用内置样式（无需写原生 CSS）；2. 自定义可复用组件类（@layer components）；3. 响应式布局（sm:/md:/lg:）；4. 动态切换样式       |
| 优先级  | 若与 Tailwind 样式冲突，需用 <code>!important</code> 强制覆盖                           | Tailwind 样式默认优先级高于原生 CSS（除非原生 CSS 加 <code>!important</code> ）                                |

## 配合使用示例（实战常用）

用 Tailwind 实现基础布局，用 `ui.add_css()` 实现精细动画 / 伪元素：

```
from nicegui import ui

# 1. 注入Tailwind自定义类（基础样式）
ui.add_head_html('''
    <style type="text/tailwindcss">
        @layer components {
            .book-card {
                @apply p-4 rounded-lg shadow-md border;
            }
        }
    </style>
''')

# 2. 用ui.add_css()实现Tailwind难以做到的动画效果
ui.add_css("""
    /* 自定义hover动画（原生CSS关键帧） */
    .book-card:hover {
        animation: shake 0.3s ease-in-out;
    }
    @keyframes shake {
        0% { transform: translateX(0); }
        25% { transform: translateX(-5px); }
        50% { transform: translateX(0); }
        75% { transform: translateX(5px); }
        100% { transform: translateX(0); }
    }
    /* 自定义伪元素（Tailwind不支持伪元素直接@apply） */
    .book-card::after {
        content: "点击查看详情";
        font-size: 12px;
    }
""")
```

```
        color: #666;
        display: block;
        margin-top: 8px;
    }
    """

# 应用样式
with ui.row().classes("gap-4"):
    ui.card().classes("book-card").label("Python编程")
    ui.card().classes("book-card").label("SQLAlchemy实战")

ui.run()
```

## 五、官方文档注意事项（重点强调）

### 1. 样式优先级问题：

NiceGUI 底层基于 Quasar Framework，Quasar 的默认样式（如 `.q-btn`）优先级较高，若用 `ui.add_css()` 覆盖默认样式，需添加 `!important`（如场景 1 中的按钮样式）。

### 2. 外部文件路径问题：

- 相对路径是相对于**主脚本文件**（如 `main.py`）的路径，而非 `ui.add_css()` 调用所在的文件；
- 若使用绝对路径，需确保路径正确（如 `C:/projects/book_system/styles/main.css`）。

### 3. `scoped` 参数的限制：

- 仅在多页面应用（`@ui.page` 定义的页面）中有效，单页面应用中无作用；
- 作用域隔离基于 CSS 选择器前缀实现，若自定义类名过于简单（如 `.box`），仍可能与其他页面的样式冲突（建议页面内使用独特的类名前缀）。

### 4. 与 `ui.add_head_html()` 的区别：

- `ui.add_css()` 是**专门用于注入 CSS** 的简化接口，自动生成 `<style>` 标签；
- `ui.add_head_html()` 是通用接口，可注入任意 HTML（包括 `<style>`、`<script>` 等），适合注入 Tailwind 样式、JS 脚本等。

### 5. 性能建议：

- 避免多次调用 `ui.add_css()` 注入重复样式，建议集中管理（如外部 CSS 文件）；
- 多页面应用中，尽量使用 `scoped=True` 减少全局样式冗余。

## 六、常见问题排查（基于官方文档 + 实战踩坑）

### 问题 1：注入的 CSS 样式不生效

- 排查 1：是否覆盖了 Quasar 默认样式？需添加 `!important`（如 `.q-btn { background: red !important; }`）；
- 排查 2：外部 CSS 文件路径是否正确？可打印路径验证（如 `import os; print(os.path.exists("styles/main.css"))`）；
- 排查 3：是否与 Tailwind 样式冲突？Tailwind 样式默认优先级更高，需用 `!important` 强制覆盖；
- 排查 4：多页面应用中，是否忘记设置 `scoped=True`？导致样式被其他页面覆盖。

问题 2：媒体查询样式不生效

- 确认 `media` 参数语法正确（如 `media="screen and (max-width: 768px)"`），注意空格和括号）；
- 避免媒体查询条件与 Tailwind 响应式前缀冲突（建议统一用一种方式实现响应式）。

问题 3：多页面样式冲突

- 为每个页面的样式添加 `scoped=True`；
- 页面内的自定义类名添加独特前缀（如 `index-`、`books-`），避免类名重复。

七、官方文档补充资源

- 官方 `ui.add_css()` 文档：[https://nicegui.io/documentation/add\\_style](https://nicegui.io/documentation/add_style)
- NiceGUI 样式系统整体说明：<https://nicegui.io/documentation/style>
- 与 Tailwind CSS 结合的官方示例：<https://nicegui.io/examples#tailwind>

NiceGUI `ui.add_css` 完全指南（结合 CSS Layers 与 CSS Variables 深度解析）

`ui.add_css` 是 NiceGUI 中注入自定义 CSS 样式的核心方法，支持原生 CSS 语法、CSS Layers（优先级控制）、CSS Variables（全局样式变量）等核心特性，是定制应用视觉风格、覆盖框架默认样式、统一全局样式规则的关键工具。其设计目标是**兼顾样式的灵活性与规范性**，既支持基础样式定制，也能应对复杂的优先级覆盖、全局变量配置场景。

一、核心定义与底层逻辑

1. 方法本质

- **签名**：`ui.add_css(css: str) -> None`
- **作用**：将传入的 CSS 字符串作为 `<style type="text/css">` 标签内容注入到应用的 HTML `<head>` 中，全局生效；
- **底层实现**：每次调用会生成一个独立的样式标签，样式优先级遵循「后注入覆盖先注入」（同优先级下），同时兼容 CSS 原生的层叠、继承规则；
- **核心价值**：
  - 覆盖框架默认样式（Quasar/NiceGUI 内置样式）；
  - 定义全局 CSS 变量，统一应用样式规则；
  - 结合 CSS Layers 精准控制样式优先级；
  - 支持所有原生 CSS 语法（媒体查询、伪类、动画等）。

2. 与其他样式方法的定位差异

| 方法 / 特性                      | <code>ui.add_css</code> 定位 | 适用场景                   |
|------------------------------|----------------------------|------------------------|
| <code>element.classes</code> | 组件级样式（添加 / 移除类名）           | 单个 / 批量组件的类名绑定，依赖预定义样式 |

| 方法 / 特性                       | <code>ui.add_css</code> 定位       | 适用场景                                                  |
|-------------------------------|----------------------------------|-------------------------------------------------------|
| <code>ui.colors</code>        | 全局颜色变量配置 (基于 Quasar CSS 变量)      | 统一主题颜色, 无需手写 CSS 规则                                   |
| <code>ui.add_head_html</code> | 注入任意 HTML (含 Tailwind 样式标签)      | 定义 Tailwind 自定义类 (需 <code>text/tailwindcss</code> 类型) |
| <code>ui.add_css</code>       | 原生 CSS 规则定义 (含 Layers/Variables) | 覆盖框架样式、全局变量配置、复杂 CSS 规则                               |

## 二、基础用法：原生 CSS 规则定义

### 1. 核心场景：基础样式定制

无需依赖 CSS Layers 或 Variables 时, 可直接定义全局样式规则, 覆盖组件默认样式或自定义类。

#### 示例 1：自定义组件类

```
from nicegui import ui

# 定义基础自定义类 (无优先级要求)
ui.add_css('''
    /* 自定义按钮类 */
    .my-btn {
        padding: 12px 24px;
        border-radius: 8px;
        border: none;
        cursor: pointer;
    }
    /* 按钮 hover 效果 */
    .my-btn:hover {
        opacity: 0.9;
        transition: opacity 0.2s;
    }
''')

# 绑定自定义类
ui.button("自定义按钮").classes("my-btn bg-blue-500 text-white")
ui.run()
```

#### 示例 2：覆盖框架默认样式 (无 Layer)

直接针对 Quasar/NiceGUI 内置类名 / 标签名写样式 (需加 `!important` 覆盖框架的 `!important` 样式) :

```
ui.add_css('''
    /* 覆盖所有按钮的默认内边距 */
    button {
        padding: 8px 16px !important;
    }
    /* 覆盖 Quasar 卡片的默认阴影 */
''')
```

```
.q-card {
    box-shadow: 0 2px 8px rgba(0,0,0,0.1) !important;
}

'''

ui.card().add(ui.label("自定义阴影的卡片"))
ui.button("覆盖默认内边距的按钮")
ui.run()
```

## 三、核心特性 1: CSS Layers 优先级控制 (NiceGUI 3.0+)

### 1. Layers 核心规则

NiceGUI 预定义 8 个 CSS Layer (优先级从低到高) :

```
theme < base < quasar < nicegui < components < utilities < overrides < quasar_importants
```

- **核心逻辑**: 高优先级 Layer 内的样式会覆盖低优先级 Layer 的样式, 无论注入顺序;
- **!important 必要性**: Quasar 大部分内置样式带 `!important`, 因此自定义样式需同时满足「高优先级 Layer + !important」才能覆盖;
- **Layer 选择原则**:
  - 组件专属样式 → `components` 层;
  - 通用工具类 → `utilities` 层;
  - 全局样式覆盖 → `overrides` 层;
  - 覆盖 Quasar 核心 `!important` 样式 → `quasar_importants` 层。

### 2. 按 Layer 定义样式的示例

#### 示例 1: `utilities` 层覆盖按钮背景色 (官方示例)

```
from nicegui import ui

# 在 utilities 层定义工具类 (高优先级, 覆盖 Quasar 按钮样式)
ui.add_css('''
    @layer utilities {
        .red-background {
            background-color: red !important; /* 必须加 !important */
        }
    }
''')

# 绑定自定义类到按钮
ui.button('Red Button').classes('red-background')
ui.run()
```

## 示例 2: 多 Layer 组合定义样式

```
ui.add_css('''
    /* 低优先级: 组件基础样式 */
    @layer components {
        .custom-card {
            border: 1px solid #e5e7eb !important;
            border-radius: 12px !important;
            padding: 16px !important;
        }
    }
    /* 高优先级: 工具类 (覆盖组件样式) */
    @layer utilities {
        .card-highlight {
            border-color: #3b82f6 !important;
            box-shadow: 0 0 0 2px rgba(59, 130, 246, 0.2) !important;
        }
    }
    /* 最高优先级: 全局覆盖 */
    @layer overrides {
        .custom-card:hover {
            transform: translateY(-2px) !important;
            transition: transform 0.2s !important;
        }
    }
''')

# 同时绑定多个 Layer 的类
ui.card().classes("custom-card card-highlight").add(ui.label("带高亮的卡片"))
ui.run()
```

## 3. 无 Layer 样式的优先级

未放入任何 Layer 的自定义 CSS 会处于「无 Layer 区域」, 优先级规则:

所有显式 Layer < 无 Layer 样式 < 行内样式

示例:

```
ui.add_css('''
    /* 无 Layer 样式, 优先级高于所有显式 Layer */
    .no-layer-btn {
        background-color: purple !important;
    }
    /* 显式 Layer 样式, 优先级更低 */
    @layer overrides {
        .layer-btn {
            background-color: green !important;
        }
    }
''')

# 绑定两个类, 最终生效 purple (无 Layer 样式)
```

```
ui.button("优先级测试").classes("layer-btn no-layer-btn")
ui.run()
```

## 四、核心特性 2: CSS Variables 全局样式配置

### 1. CSS Variables 核心规则

- NiceGUI 暴露了全局可配置的 CSS 变量（当前官方支持的核心变量）：

| 变量名                                    | 默认值  | 作用                |
|----------------------------------------|------|-------------------|
| <code>--nicegui-default-padding</code> | 1rem | 全局默认内边距           |
| <code>--nicegui-default-gap</code>     | 1rem | 全局默认间距（flex/grid） |

- 自定义 CSS 变量需定义在 `:root` 选择器下（全局生效），也可定义在特定类 / 标签下（局部生效）；
- 变量可被后续样式规则引用，支持动态修改（通过 JS）。

### 2. 内置变量覆盖示例（官方示例）

```
from nicegui import ui

# 覆盖 NiceGUI 内置 CSS 变量
ui.add_css('''
    :root {
        --nicegui-default-padding: 0.5rem; /* 缩小默认内边距 */
        --nicegui-default-gap: 3rem;      /* 放大默认间距 */
    }
''')

# 卡片内的元素会自动应用新的 padding/gap
with ui.card():
    ui.label('small padding') # 内边距 0.5rem
    ui.label('large gap')     # 与上一个标签的间距 3rem

ui.run()
```

### 3. 自定义 CSS 变量示例

```
from nicegui import ui

# 定义自定义全局变量 + 基于变量的样式规则
ui.add_css('''
    :root {
        /* 自定义颜色变量 */
        --app-primary: #165dff;
        --app-secondary: #6b7280;
        /* 自定义尺寸变量 */
        --app-card-width: 320px;
        --app-card-radius: 8px;
    }
''')
```

```

/* 引用自定义变量的样式规则 */
@layer components {
    .app-card {
        width: var(--app-card-width) !important;
        border-radius: var(--app-card-radius) !important;
        border: 1px solid var(--app-secondary/20) !important;
        background-color: white !important;
    }
    .app-btn {
        background-color: var(--app-primary) !important;
        color: white !important;
        padding: 8px 16px !important;
        border-radius: var(--app-card-radius) !important;
    }
}

'''

# 应用自定义变量样式
ui.card().classes("app-card").add(ui.label("基于自定义变量的卡片"))
ui.button("基于自定义变量的按钮").classes("app-btn")
ui.run()

```

## 4. 动态修改 CSS 变量

结合 `ui.run_javascript` 可运行时修改变量，实现动态样式调整：

```

from nicegui import ui

# 初始变量配置
ui.add_css('''
:root {
    --app-primary: #165dff;
}
@layer utilities {
    .app-btn {
        background-color: var(--app-primary) !important;
    }
}
''')

def change_primary_color():
    # 动态修改 CSS 变量
    ui.run_javascript('''
        document.documentElement.style.setProperty('--app-primary', '#f53f3f');
    ''')

ui.button("动态修改颜色", classes="app-btn", on_click=change_primary_color)
ui.run()

```

## 五、高级用法：组合特性与复杂场景

### 1. 响应式样式（媒体查询）

```
ui.add_css('''
    @layer utilities {
        .responsive-card {
            width: 320px !important;
            padding: 16px !important;
        }
        /* 大屏适配 */
        @media (min-width: 768px) {
            .responsive-card {
                width: 480px !important;
                padding: 24px !important;
            }
        }
        /* 暗色模式适配 */
        @media (prefers-color-scheme: dark) {
            .responsive-card {
                background-color: #1f2937 !important;
                color: white !important;
            }
        }
    }
''')

ui.card().classes("responsive-card").add(ui.label("响应式卡片"))
ui.run()
```

### 2. 伪类 / 伪元素 + Layers/Variables

```
ui.add_css('''
    :root {
        --tooltip-bg: #1f2937;
        --tooltip-text: #f9fafb;
    }

    @layer components {
        .tooltip {
            position: relative !important;
            cursor: pointer !important;
        }
        /* 伪元素实现提示框 */
        .tooltip::after {
            content: attr(data-tooltip) !important;
            position: absolute !important;
            top: -30px !important;
            left: 50% !important;
            transform: translateX(-50%) !important;
            background-color: var(--tooltip-bg) !important;
        }
    }
''')
```

```

        color: var(--tooltip-text) !important;
        padding: 4px 8px !important;
        border-radius: 4px !important;
        font-size: 12px !important;
        opacity: 0 !important;
        transition: opacity 0.2s !important;
        pointer-events: none !important;
    }
    .tooltip:hover::after {
        opacity: 1 !important;
    }
}
'''

# 绑定伪元素样式 + 自定义属性
ui.button("Hover for Tooltip", attrs={"data-tooltip": "Hello world!"}).classes("tooltip")
ui.run()

```

### 3. 与 `ui.colors` 配合使用

`ui.colors` 配置的 Quasar 颜色变量可被 `ui.add_css` 引用, 实现样式统一:

```

from nicegui import ui

# 1. 配置全局颜色变量
ui.colors(primary="#165dff", secondary="#6b7280")

# 2. 在 CSS 中引用 Quasar 颜色变量
ui.add_css('''
    @layer utilities {
        .primary-btn {
            background-color: var(--q-primary) !important;
            color: white !important;
        }
        .secondary-text {
            color: var(--q-secondary) !important;
        }
    }
''')

ui.button("主色按钮").classes("primary-btn")
ui.label("次要文本").classes("secondary-text")
ui.run()

```

## 六、关键注意事项

### 1. !important 的使用规则

- 覆盖 Quasar 样式：必须加 `!important`（无论是否在高优先级 Layer）；
- 同 Layer 内样式：后定义的 `!important` 覆盖先定义的；
- 不同 Layer 内样式：高优先级 Layer 的 `!important` 覆盖低优先级的。

### 2. 样式注入顺序

- 多次调用 `ui.add_css` 会按顺序生成样式标签，同优先级（同 Layer / 无 Layer）下，后注入的样式覆盖先注入的；
- 建议将全局变量、基础 Layer 样式先注入，覆盖类、工具类后注入。

### 3. 与 Tailwind CSS 的配合

- `ui.add_css` 用于原生 CSS 规则（Layers/Variables），`ui.add_head_html` 用于 Tailwind 自定义类（需 `<style type="text/tailwindcss">`）；
- 自定义 CSS 变量可通过 `tailwind.config` 扩展为 Tailwind 类：

```
ui.add_head_html(``  
  <script>  
    tailwind.config = {  
      theme: {  
        extend: {  
          colors: {  
            app: 'var(--app-primary)',  
          },  
        }  
      }  
    }  
  </script>  
``)
```

### 4. 调试技巧

- 浏览器开发者工具 → Elements → Styles：查看样式是否生效，以及被哪个规则覆盖；
- 查看 Layer 优先级：Styles 面板中会标注样式所属的 Layer（如 `@layer utilities`）；
- 检查 CSS 变量：Elements → Computed → 找到变量名（如 `--nicegui-default-padding`），查看当前值。

### 5. 性能与维护

- 避免定义过多零散的样式规则，建议按功能分类（如 `variables.css`、`components.css`、`utilities.css`）拆分后注入；
- 复杂动画、大量样式规则建议通过外部 CSS 文件引入（`ui.add_head_html('<link rel="stylesheet" href="style.css">')`）。

# ui.add\_css 方法全解析（结合 NiceGUI 3.0+ CSS Layers）

`ui.add_css` 是 NiceGUI 中用于注入自定义 CSS 样式的核心方法，支持原生 CSS 语法，且在 3.0.0 版本后适配了 CSS Layers 特性，可精准控制样式优先级，是定制 UI 外观、覆盖框架默认样式（如 Quasar/NiceGUI 内置样式）的关键手段。

## 一、基本定义与核心作用

### 1. 方法本质

`ui.add_css` 用于将自定义 CSS 代码嵌入到 NiceGUI 应用的 HTML 头部（`<style>` 标签内），作用域覆盖整个应用，可定义全局样式、自定义类名、覆盖框架默认样式，或结合 CSS Layers 控制样式优先级。

### 2. 方法签名（简化版）

```
def add_css(self, css: str) -> None:
    """
    向应用添加自定义 CSS 样式。

    参数:
        css: 字符串格式的 CSS 代码（支持完整 CSS 语法，包括 @layer、@media 等规则）
    返回:
        None
    """
```

## 二、基础用法

### 1. 全局自定义类（无 CSS Layer）

适用于基础样式定制，无需处理优先级，直接定义类名即可。

```
from nicegui import ui

# 定义全局自定义 CSS 类
ui.add_css('''
/* 自定义按钮样式 */
.my-custom-btn {
    background-color: #2563eb;
    color: white;
    border-radius: 8px;
    padding: 12px 24px;
    border: none;
    cursor: pointer;
}
/* 按钮hover效果 */
.my-custom-btn:hover {
    background-color: #1d4ed8;
}
''')
```

```
'''

# 绑定自定义类
ui.button('Custom Button').classes('my-custom-btn')

ui.run()
```

2. 覆盖元素默认样式 (无 CSS Layer)

直接针对 HTML 标签或框架内置类名写样式（优先级可能不足，需结合 Layer/!important）。

```
ui.add_css('''
    /* 覆盖 NiceGUI 按钮的默认内边距 */
    button {
        padding: 8px 16px !important; /* 需 !important 覆盖框架默认 */
    }
''')
ui.button('Default Button') # 样式会被上述 CSS 覆盖
```

三、核心特性：适配 CSS Layers (NiceGUI 3.0+)

1. CSS Layers 核心逻辑

NiceGUI 预定义了 8 个 CSS Layer（优先级从低到高）：

theme < base < quasar < nicegui < components < utilities < overrides < quasar\_important

- 高优先级 Layer 内的样式会覆盖低优先级 Layer 的样式；
- Quasar 大部分样式带 !important，需在高优先级 Layer 中用 !important 才能覆盖；
- 未放入 Layer 的自定义 CSS 会处于「无 Layer 区域」，优先级高于所有显式 Layer（但低于 !important）。

2. 按 Layer 定义样式 (覆盖框架样式)

根据自定义样式的用途选择对应 Layer，核心场景是覆盖 Quasar/NiceGUI 的默认样式：

| Layer 名称         | 适用场景                       | 优先级 |
|------------------|----------------------------|-----|
| components       | 组件专属样式（如自定义按钮、卡片）          | 中高  |
| utilities        | 通用工具类（如间距、颜色工具）            | 中高  |
| overrides        | 全局样式覆盖                     | 高   |
| quasar_important | 覆盖 Quasar 带! important 的样式 | 最高  |

示例 1：用 utilities 层覆盖按钮背景色

```
from nicegui import ui

# 在 utilities 层定义工具类（覆盖 Quasar 样式需 !important）
ui.add_css('''
    @layer utilities {
```

```

        .red-background {
            background-color: red !important; /* 必须加 !important 覆盖 Quasar */
        }
    }
    ''')

# 绑定自定义类到按钮
ui.button('Red Button').classes('red-background')

ui.run()

```

## 示例 2: 用 `components` 层定制卡片样式

```

ui.add_css('''
    @layer components {
        .custom-card {
            border: 2px solid #6366f1 !important;
            border-radius: 12px !important;
            box-shadow: 0 4px 12px rgba(0,0,0,0.1) !important;
            padding: 20px !important;
        }
    }
    ''')

ui.card().classes('custom-card').add(ui.label('Custom Card Content'))

```

## 示例 3: 用 `quasar_importants` 层覆盖 Quasar 核心样式

```

# 覆盖 Quasar 输入框的默认边框样式（最高优先级）
ui.add_css('''
    @layer quasar_importants {
        .q-input {
            border: 1px solid #ef4444 !important;
        }
    }
    ''')

ui.input('Input with Red Border')

```

## 3. 多层样式定义

可在一次 `ui.add_css` 中定义多个 Layer 的样式，按优先级自动生效：

```

ui.add_css('''
    /* 组件层：自定义按钮基础样式 */
    @layer components {
        .my-btn {
            padding: 10px 20px !important;
        }
    }
    /* 工具层：按钮颜色工具类（优先级更高） */
    @layer utilities {

```

```

        .btn-primary {
            background-color: #3b82f6 !important;
            color: white !important;
        }
    }
    /* 覆盖层: 强制修改 hover 样式 (优先级更高) */
    @layer overrides {
        .my-btn:hover {
            transform: scale(1.05) !important;
        }
    }
}
'''

ui.button('Multi-Layer Button').classes('my-btn btn-primary')

```

## 四、高级用法

### 1. 响应式样式

结合 CSS `@media` 规则，实现响应式布局 / 样式：

```

ui.add_css('''
    @layer utilities {
        .responsive-btn {
            padding: 8px 16px !important;
        }
        /* 大屏下放大按钮 */
        @media (min-width: 768px) {
            .responsive-btn {
                padding: 12px 24px !important;
                font-size: 18px !important;
            }
        }
    }
}
'''

ui.button('Responsive Button').classes('responsive-btn')

```

### 2. 伪类 / 伪元素

支持所有 CSS 伪类 (`:hover` / `:focus`)、伪元素 (`::before` / `::after`)：

```

ui.add_css('''
    @layer components {
        .tooltip-btn::after {
            content: attr(data-tooltip) !important;
            position: absolute !important;
            top: -30px !important;
            left: 50% !important;
            transform: translateX(-50%) !important;
            background-color: #1f2937 !important;
            color: white !important;
        }
    }
}
'''

```

```

padding: 4px 8px !important;
border-radius: 4px !important;
font-size: 12px !important;
opacity: 0 !important;
transition: opacity 0.2s !important;
}
.tooltip-btn:hover::after {
    opacity: 1 !important;
}
}
'''

# 绑定伪元素样式+自定义属性
ui.button('Hover for Tooltip', attrs={'data-tooltip': 'Hello world!'}).classes('tooltip-btn
relative')

```

### 3. 动态修改 CSS

结合 `ui.run_javascript` 动态更新样式（补充 `ui.add_css` 的静态限制）：

```

from nicegui import ui

# 初始 CSS
ui.add_css('''
    @layer utilities {
        .dynamic-btn {
            background-color: blue !important;
        }
    }
''')

def change_color():
    # 动态修改 CSS 样式
    ui.run_javascript('''
        // 获取样式表并修改
        const styleSheet = document.styleSheets[document.styleSheets.length - 1];
        styleSheet.replaceSync(`
            @layer utilities {
                .dynamic-btn {
                    background-color: green !important;
                }
            }
        `);
    ''')

ui.button('Dynamic Color Button').classes('dynamic-btn')
ui.button('Change Color', on_click=change_color)

ui.run()

```

## 五、注意事项

1. **!important 必要性**: Quasar 大部分内置样式带 `!important`，因此覆盖时自定义样式必须加 `!important`（即使放在高优先级 Layer）；
2. **Layer 选择原则**: 优先用 `components / utilities`，仅需全局覆盖时用 `overrides`，极端场景（覆盖 Quasar 核心 `!important` 样式）用 `quasar-important`；
3. **样式作用域**: `ui.add_css` 是全局生效的，如需组件级隔离，可结合元素的 `classes` 绑定唯一类名；
4. **加载顺序**: 多次调用 `ui.add_css` 会按顺序注入样式，后注入的样式（同 Layer）会覆盖先注入的；
5. **语法校验**: CSS 语法错误会导致样式失效，建议先在浏览器调试工具验证 CSS 代码。